Week 3

COGS 108 SP 2026

Due dates

- GitHub ID & Group Signup: Thur (04/16)
- D2: Friday (04/17)

Fermi Estimation

- used when an exact answer is hard or impossible to get quickly
- helps us make a reasonable estimate from limited information
- break a big question into smaller assumptions
- combine rough estimates to get an approximate answer
- Example:
    - how many piano tuners are there in Chicago?
    - how many coffee cups are used on campus in one day?

Data Structures

Structured data

- can be stored in database SQL
- tables with rows and columns
- requires a relational key
- 5-10% of all data

Semi-structured

- doesn't reside in a relational database
- but still has organizational properties
- CSV, XML, JSON

Unstructured

- non-tabular data
- 80% of the world's data by bytes
- No relationships inherent in the data
- images, text, audio, videos

Clean vs Tidy Data

- Tidy refers to data shape and structure
- Clean refers to data accuracy and consistency
- A dataset can be tidy but still not be clean
- A dataset can be clean but still be hard to analyze if it is not tidy

Relational Keys

- Data is often split across multiple tables
- We use keys to connect related records
- Example: one table for patients, one for visits
    - A shared ID lets us match the right visit to the right patient

users

user_id	email	name
10	sadio@example.com	Sadio
11	mo@example.com	Mohamed
12	rinsola@example.com	Rinsola
13	amalie@example.com	Amalie

orders

order_no	user_id	product_sku
93	11	123
94	11	789
95	13	789
96	10	101

A row can only be added or updated in the **orders** table if the value in **orders.user_id** matches an existing user ID in the **users** table.

This type of **database rule** is called a **foreign key constraint**.

Wide vs Long Data

- The same data can be organized in different shapes
- Wide form: more columns, fewer rows
- Long form: fewer columns, more rows

Wide Format

Team	Points	Assists	Rebounds
A	88	12	22
B	91	17	28
C	99	24	30
D	94	28	31

Long Format

Team	Variable	Value
A	Points	88
A	Assists	12
A	Rebounds	22
B	Points	91
B	Assists	17
B	Rebounds	28
C	Points	99
C	Assists	24
C	Rebounds	30
D	Points	94
D	Assists	28
D	Rebounds	31

What is Data Wrangling?

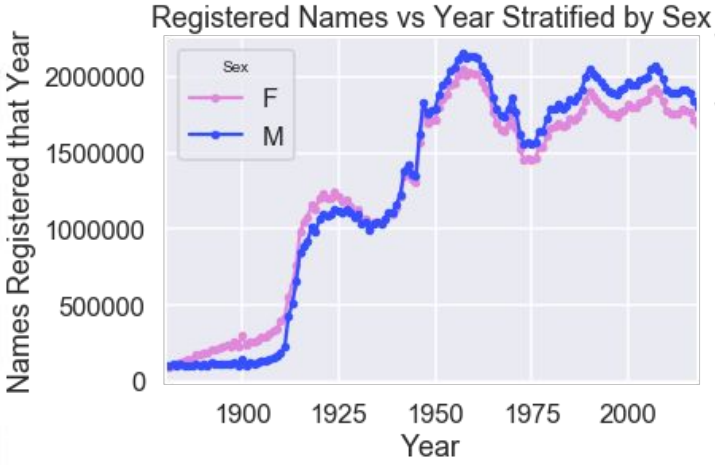
Data wrangling in python deals with the below functionalities:

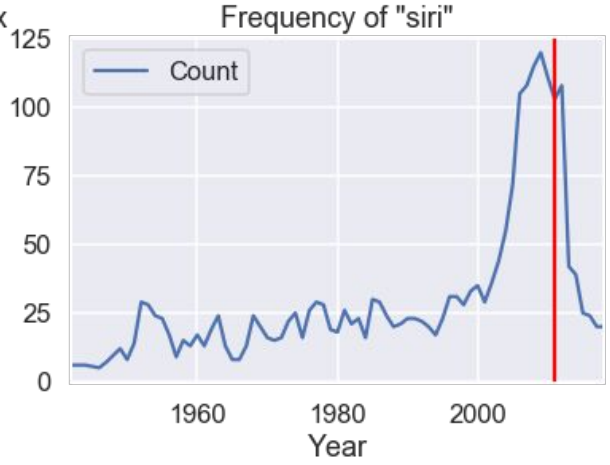
- Data exploration
  - In this process, the data is studied, analyzed and understood by visualizing representations of data.
- Dealing with missing values
  - Most of the datasets having a vast amount of data contain missing values of NaN, they are needed to be taken care of by replacing them with mean, mode, the most frequent value of the column or simply by dropping the row having a NaN value.
- Reshaping data
  - In this process, data is manipulated according to the requirements, where new data can be added or pre-existing data can be modified.
- Filtering data
  - Some times datasets are comprised of unwanted rows or columns which are required to be removed or filtered

Pandas is really useful!

	Name	Sex	Count	Year
0	Mary	F	7065	1880
1	Anna	F	2604	1880
2	Emma	F	2003	1880
...	...	...	...	...
1957043	Zyrie	M	5	2018
1957044	Zyron	M	5	2018
1957045	Zzyzx	M	5	2018

1957046 rows × 4 columns





It converts python into a usable (and good!) data analysis tool

Pandas has terrible error messages

	Timestamp	Name	Sex	Age
0	10/15/2019 21:49:38	samuel	M	24
1	10/16/2019 9:07:31	aditi	F	22
2	10/16/2019 9:07:34	hanyang	M	21
...	...	...	...	...
24	10/16/2019 16:08:45	amy	F	20
25	10/16/2019 16:08:46	sheila	F	21
26	10/16/2019 16:09:15	thomas	M	23

```
students['name']
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
~/anaconda3/lib/python3.7/site-packages/pandas/core/indexes/base.py in get_loc(self, key, method, tolerance)
   2656             try:
-> 2657                 return self._engine.get_loc(key)
   2658             except KeyError:

pandas/_libs/index.pyx in pandas._libs.index.IndexEngine.get_loc()

pandas/_libs/index.pyx in pandas._libs.index.IndexEngine.get_loc()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'name'

During handling of the above exception, another exception occurred:

KeyError                                  Traceback (most recent call last)
<ipython-input-27-ae454297f350> in <module>()
----> 1 students['name']

~/anaconda3/lib/python3.7/site-packages/pandas/core/frame.py in __getitem__(self, key)
   2925             if self.columns.nlevels > 1:
   2926                 return self._getitem_multilevel(key)
-> 2927             indexer = self.columns.get_loc(key)
   2928             if is_integer(indexer):
   2929                 indexer = [indexer]

~/anaconda3/lib/python3.7/site-packages/pandas/core/indexes/base.py in get_loc(self, key, method, tolerance)
   2657                 return self._engine.get_loc(key)
   2658             except KeyError:
-> 2659                 return self._engine.get_loc(self._maybe_cast_indexer(key))
   2660         indexer = self.get_indexer([key], method=method, tolerance=tolerance)
   2661         if indexer.ndim > 1 or indexer.size > 1:

pandas/_libs/index.pyx in pandas._libs.index.IndexEngine.get_loc()

pandas/_libs/index.pyx in pandas._libs.index.IndexEngine.get_loc()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

pandas/_libs/hashtable_class_helper.pxi in pandas._libs.hashtable.PyObjectHashTable.get_item()

KeyError: 'name'
```

Pandas has unfriendly documentation

`DataFrame.rename`(*self, mapper=None, index=None, columns=None, axis=None, copy=True, inplace=False, level=None, errors='ignore'*) [source]

Alter axes labels.

Function / dict values must be unique (1-to-1). Labels not contained in a dict / Series will be left as-is. Extra labels listed don't throw an error.

See the user guide for more.

Parameters:	**mapper** : *dict-like or function* Dict-like or functions transformations to apply to that axis' values. Use either `mapper` and `axis` to specify the axis to target with `mapper`, or `index` and `columns`. **index** : *dict-like or function* Alternative to specifying axis (`mapper, axis=0` is equivalent to `index=mapper`). **columns** : *dict-like or function* Alternative to specifying axis (`mapper, axis=1` is equivalent to `columns=mapper`). **axis** : *int or str* Axis to target with `mapper`. Can be either the axis name ('index', 'columns') or number (0, 1). The default is 'index'. **copy** : *bool, default True* Also copy underlying data. **inplace** : *bool, default False* Whether to return a new DataFrame. If True then value of copy is ignored. **level** : *int or level name, default None* In case of a MultiIndex, only rename labels in the specified level. **errors** : *{'ignore', 'raise'}, default 'ignore'* If 'raise', raise a *KeyError* when a dict-like *mapper*, *index*, or *columns* contains labels that are not present in the Index being transformed. If 'ignore', existing keys will be renamed and extra keys will be ignored.
Returns:	DataFrame DataFrame with the renamed axis labels.
Raises:	KeyError If any of the labels is not found in the selected axis and "errors='raise'".

Also, there are typically many ways to do the same thing in pandas.

What's a DataFrame?

Data Frame: two-dimensional table of data.

All columns are the same type (but not rows).

Every row and every column has a label.

We call the set of row labels the Index of a DataFrame

Year	Candidate	Party	%	Result
2008	Obama	Democratic	52.9	win
2008	McCain	Republican	45.7	loss
2012	Obama	Democratic	51.1	win
2012	Romney	Republican	47.2	loss
2016	Clinton	Democratic	48.2	loss
2016	Trump	Republican	46.1	win

Index

What's a Series?

Series: one-dimensional sequence of data.

Usually created by taking a single column from a Data Frame.

Index

```
0      Obama
1    McCain
2      Obama
3    Romney
4   Clinton
5     Trump
Name: Candidate
```

Why is this important?

Most pandas methods work differently between Data Frames and Series.

The documentation will tell you what type of object the method is for.

pandas.DataFrame.sort_values¶

`DataFrame.sort_values(self, by, axis=0, ascending=True, inplace=False, kind='quicksort', na_position='last')` [source]

Sort by the values along either axis.

by : *str or list of str*

Name or list of names to sort by.

- if *axis* is 0 or *'index'* then *by* may contain index levels and/or column labels
- if *axis* is 1 or *'columns'* then *by* may contain column levels and/or index labels

Changed in version 0.23.0: Allow specifying index or column level names.

pandas.Series.sort_values¶

`Series.sort_values(self, axis=0, ascending=True, inplace=False, kind='quicksort', na_position='last')` [source]

Sort by the values.

Sort a Series in ascending or descending order by some criterion.

Parameters:

axis : *{0 or 'index'}, default 0*

Axis to direct sorting. The value 'index' is accepted for compatibility with DataFrame.sort_values.

ascending : *bool, default True*

If True, sort values in ascending order, otherwise descending.

inplace : *bool, default False*

If True, perform operation in-place.

kind : *{'quicksort', 'mergesort' or 'heapsort'}, default 'quicksort'*

Choice of sorting algorithm. See also `numpy.sort()` for more information. 'mergesort' is the only stable algorithm.

na_position : *{'first' or 'last'}, default 'last'*

Argument 'first' puts NaNs at the beginning, 'last' puts NaNs at the end.

Why is this important?

df.sort_values(…)

df['names'].sort_values(…)

pd.read_csv(…)

pandas.DataFrame.sort_values¶

`DataFrame.sort_values(self, by, axis=0, ascending=True, inplace=False, kind='quicksort', na_position='last')` [source]

Sort by the values along either axis.

by : *str or list of str*

Name or list of names to sort by.

- if *axis* is 0 or *'index'* then *by* may contain index levels and/or column labels
- if *axis* is 1 or *'columns'* then *by* may contain column levels and/or index labels

Changed in version 0.23.0: Allow specifying index or column level names.

pandas.Series.sort_values¶

`Series.sort_values(self, axis=0, ascending=True, inplace=False, kind='quicksort', na_position='last')` [source]

Sort by the values.

Sort a Series in ascending or descending order by some criterion.

pandas.read_csv

`pandas.read_csv(filepath_or_buffer, sep=', ', delimiter=None, header='infer', names=None, index_col=None, usecols=None, squeeze=False, prefix=None, mangle_dupe_cols=True, dtype=None, engine=None, converters=None, true_values=None, false_values=None, skipinitialspace=False, skiprows=None, nrows=None, na_values=None, keep_default_na=True, na_filter=True, verbose=False, skip_blank_lines=True, parse_dates=False, infer_datetime_format=False, keep_date_col=False, date_parser=None, dayfirst=False, iterator=False, chunksize=None, compression='infer', thousands=None, decimal=b'.', lineterminator=None, quotechar='"', quoting=0, escapechar=None, comment=None, encoding=None, dialect=None, tupleize_cols=None, error_bad_lines=True, warn_bad_lines=True, skipfooter=0, doublequote=True, delim_whitespace=False, low_memory=True, memory_map=False, float_precision=None)` [source]

Read CSV (comma-separated) file into DataFrame

How to read pandas documentation

Skip the table of method parameters and look at the examples.

Copy example, then modify it to work for your notebook.

If needed, refer back to the method parameters for fine-tuning.

(The method in the picture on the right solves Q2.)

pandas.read_csv¶

`pandas.read_csv(filepath_or_buffer: Union[str, pathlib.Path, IO[~AnyStr]], sep=',', delimiter=None, header='infer', names=None, index_col=None, usecols=None, squeeze=False, prefix=None, mangle_dupe_cols=True,`

Examples

```
>>> pd.read_csv('data.csv')  # doctest: +SKIP
```

Read a comma-separated values (csv) file into DataFrame.

Also supports optionally iterating or breaking of the file into chunks.

Additional help can be found in the online docs for IO Tools.

filepath_or_buffer : *str, path object or file-like object*

Any valid string path is acceptable. The string could be a URL. Valid URL schemes include http, ftp, s3, and file. For file URLs, a host is expected. A local file could be: file://localhost/path/to/table.csv.

If you want to pass in a path object, pandas accepts any `os.PathLike`.

By file-like object, we refer to objects with a `read()` method, such as a file handler (e.g. via builtin `open` function) or `StringIO`.

sep : *str, default ','*

Delimiter to use. If sep is None, the C engine cannot automatically detect the separator, but the Python parsing engine can, meaning the latter will be used and automatically detect the separator by Python's builtin sniffer tool, `csv.Sniffer`. In addition, separators longer than 1 character and different from `'\s+'` will be interpreted as regular expressions and will also force the use of the Python parsing engine. Note that regex delimiters are prone to ignoring quoted data. Regex example: `'\r\t'`.

delimiter : *str, default `None`*

Alias for sep.

header : *int, list of int, default 'infer'*

Row number(s) to use as the column names, and the start of the data. Default behavior is to infer the column names: if no names are passed the behavior is identical to `header=0` and column names are inferred from the first line of the file, if column names are passed explicitly then the behavior is identical to `header=None`. Explicitly pass `header=0` to be able to replace existing names. The header can be a list of integers that specify row locations for a multi-index on the columns e.g. [0,1,3]. Intervening rows that are not specified will be skipped (e.g. 2 in this example is skipped). Note that this parameter ignores commented lines and empty lines if

Basic Pandas Operations

- remove rows with missing data:
  `scores.dropna()`
- select rows and/or columns:
  `scores.loc[10:25, [column1, column2, column3]]`
- group and summarize:
  `scores.groupby(column).final.mean()`
- merge on a shared ID:
  `pd.merge(roster, grades, on='column_name')`
- get summary statistics:
  `scores.describe()`

Finally: don’t use loops

If you find yourself trying to write a for/while loop when working with pandas, you’re almost definitely doing it wrong.

Look for the right pandas method. And ask your friend + staff for help.

Quiz Time

Your time to …

- Work on D2